Module 2: Imperative Languages—Data Control and Data Types

Topics

1. Data Control
2. Scalar Data Types
3. Structured Data Types
4. Pointers and References

I. Data Control

Before we discuss the issues that relate to the different kinds of data types, we begin with a variety of issues that concern data types in general. These issues are sometimes referred to as *data control*. Many of these data-control issues are central to the design of a compiler's symbol table.

In module 1, we discussed the syntax of identifiers or names, which in most languages consist of some combination of letters and digits and perhaps also a few special characters—underscores being the most common. The identifier token is unique among the tokens of programming languages in that it is the only one that does not have a predefined meaning. Most lexemes that are identifiers are given meaning through declarations, which are then stored in the symbol table. There are, however, some lexemes that syntactically are identifiers that do have predefined meanings. These lexemes include reserved words, keywords, and predefined names.

Languages vary greatly with regard to the number of reserved words they contain. Making a name a reserved word emphasizes the importance of the concept associated with that word. Among popular imperative languages, Ada has the most

reserved words. At the other extreme is LISP, with none. Deciding which features of the language are important enough to warrant reserved words is a key decision that influences how easy a language is to learn.

When we examine particular language features, we may occasionally consider the number of reserved words associated with that feature. Concurrency is a good example. Ada's implementation of concurrency involves a dozen or more reserved words, whereas Java has only one. In module 4, we will examine the implications of that design decision.

A. Binding

A commonly used term in programming language design is *binding*. Among the important issues related to binding is when it occurs. These binding times are referred to as either *static binding* or *dynamic binding*

We will examine two kinds of binding, the binding of variables to types and the binding of variables to memory addresses.

Variable-to-Type Binding

We begin with a discussion of the binding of variables to types. The first variation among languages is whether variable declarations are required or not. We first consider languages that do not require variable declarations.

Implicit Declarations

In some cases, the type of the variable is determined by using a naming convention that implicitly specifies the type. Such *implicit declarations* are now rare among compiled languages, but Fortran still uses such a convention. In Fortran, variables that begin with the letters *I* through *N*, if not otherwise declared, are taken to be integers. This convention is based on the practice in mathematics of using letters, such as *i* and *j*, as subscripts.

A criterion that we should use to judge the quality of a language design decision, one that we did not mention in module 1, is how that decision influences error detection. All programmers recognize that the earlier in the software development process an error occurs, the easier it is to fix. In particular, compilation errors are much easier to fix than logic errors because logic errors are never reported and

must be uncovered by debugging. Although allowing implicit declarations makes programs easier to write because one is spared having to declare variables, and perhaps easier to read because its type is apparent from its name, there is a significant disadvantage, which involves the late detection of errors. Suppose one mistakenly writes the following line of Fortran code:

```
ICOUNT = ICOUMT + 1
```

A typographical mistake, like this one, is easy to make. In a language that requires explicit declaration of all variables, this line would produce a compilation error, specifically the static semantic error *undeclared variable*. In Fortran, however, this mistake results in a logic error. Because logic errors are so much more costly to fix than compilation errors, this language design decision can have an adverse effect by increasing the cost of writing such Fortran programs.

Another drawback to such implicit typing is that it does not extend well. If a language allows user-defined types, there is no easy way to have it apply to such types. Because of these two drawbacks, most modern compiled languages do not use naming conventions as a way to specify type.

Another way the type of a variable can be determined when it is implicitly declared is referred to as *type inference*. Although rarely used in compiled languages, there is one instance in Ada where type inference is used—the Ada for statement, as illustrated in the following example:

```
for (I in 1..10)
  Put(I);
```

Loop-control variables inside for loops are implicitly declared. In fact, if the variable I is declared in an outer block, it is a different variable from the I inside the loop. The reason for making these declarations implicit was so that the scope of the variable could be guaranteed to not extend beyond the body of the loop. In this case, it is not the name that is used to conclude that I is an integer but the type of

the constants, which specify the lower and upper bounds of the loop-control variable. Although Ada prohibits any real types in this context, it allows any discrete type, which includes enumeration types.

In both the Fortran and Ada examples of implicit declarations, the binding is static—it happens at compile time. Although implicit declarations are rare in compiled languages, they are common among interpreted languages. Because interpreted languages are not compiled, the binding must be dynamic. Some interpreted languages, such as LISP, are essentially typeless. In such languages, no variable-type binding ever occurs.

Explicit Declarations

Next, let's consider languages that have explicit declarations. As we have seen in compiled languages, the variable-type binding is generally static even when variables are implicitly declared. The same is certainly true for declared variables. There is, however, a case where binding becomes dynamic—in object-oriented languages. In such contexts, we generally refer to objects rather than variables. Consider the following Java declaration:

```
Object obj = newObject();
```

In this situation, the type of obj is not determined until run time, so the binding is dynamic. What can be checked at compile time is that the method newObject, sometimes called a *factory method*, returns a type that is either the declared type or any type derived from it.

Variable-to-Storage Binding

The next kind of binding that we consider is the binding of a variable to a memory address. Unlike variable-type binding, the division between static and dynamic binding does not sufficiently distinguish the various cases because there are cases in which some aspect of the address can be determined at compile time but another aspect at run time. So, we consider the various cases along a spectrum from most static to most dynamic.

The first case is generally referred to simply as *static variables*. All variables in the early versions of Fortran fall into this category. Global variables and local variables prefaced with the storage class specifier static in C or C++ are other examples of static variables. The addresses of such variables are determined at compile time, but even in this case, we must qualify the exactness of this determination. Most compilers produce what is called *relocatable object code*, which as the name suggests, can be loaded at different locations in memory. Some compilers produce what are sometimes called *core images*, which contain absolute addresses, but because of the inflexibility of such object code, it is not a common practice.

Furthermore, in a machine that has virtual memory, we can never be sure of the absolute address of a variable because it can change throughout the execution of a program.

One advantage of a language that allows only static variables is that once the program is compiled, we can with certainty know how much memory the program will require when it runs, because the memory requirements of such programs are constant—O(1). As soon as a language allows any more dynamic kind of binding, we introduce the possibility of programs that require an unbounded amount of memory when they run.

Any programming language that allows recursion must allow a more dynamic form of variable-storage binding. One reason is that the local variables of any recursive subprogram can have an unlimited number of instances—one for each activation of the subprogram. Although we observed in module 1 that recursion properly belongs to the domain of declarative languages, most modern imperative languages support it. To be able to support recursion, it is necessary for programming languages to allow stack-dynamic variables. The name suggests that such variables are allocated on the compiler's run-time stack.

It is possible to some degree to compute the address of a stack-dynamic variable at compile time. Its address can be computed relative to the beginning of the activation record in which it resides. The code below illustrates how the relative addresses of such variables can be computed in a Java method.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
void m()
{
    int i;
    double d;
    byte b;
    ...
}
```

The actual address is the sum of the address of the activation record, which can only be determined at run time, plus the relative address inside the activation record, which can be computed at compile time.

In languages, such as C, C++, Java, and C#, that do not allow nested subprograms, only the local variables of the current subprogram can be accessed, so the address relative to the beginning of the subprogram is all that must be determined at compile time. But in languages that allow nested subprograms, such as Algol, Pascal, and Ada, another component of address information must be determined at compile time—which activation record the local variable is in. The Ada subprogram shown below illustrates this point.

Move your mouse over the highlighted part of the program to see an explanation of that part.

```
procedure One is
    X: Integer;
    procedure Two is
       procedure Three is
       begin
           X := 1;
       end Three;
    begin
        null;
    end Two;
begin
    null;
end One;
```

When we discuss scope rules later in this module, we will elaborate on why local variables of other subprograms are accessible when subprograms are nested. The distance, measured in levels of nesting, is still not sufficient information to locate the actual activation record. In module 3, we will return to this issue and further explain what other information must be maintained at run time to locate the actual activation record.

The most dynamic binding between variables and memory addresses is *heap-dynamic* binding. The memory for variables bound in this fashion comes from the heap. No address information whatsoever can be determined for such variables at compile time. It is done exclusively at run time.

There are two varieties of such bindings. The first variety is explicit and involves a specific instruction coded by the programmer to cause memory allocation. The reserved word new is used in several imperative languages, including C++, Ada, Java, and C#, to request such explicit allocation. Languages that provide explicit heap allocation have either pointers or references associated with this kind of allocation. Some have both. Pointers and references introduce the possibility of aliasing in the language.

Historically, languages that had explicit heap allocation required explicit heap deallocation. This requirement exists in Pascal, C, and C++. Explicit deallocation in C++, for example, is accomplished with the reserved word delete. Ada began a movement toward implicit deallocation despite explicit allocation. Ada compilers are given the option to perform what is usually called *automatic garbage collection*. In Java and C#, automatic garbage collection is required.

Automatic garbage collection was first included in languages such as LISP that use *implicit heap-dynamic* binding—the other variety of heap dynamic binding. In such languages, the programmer never explicitly requests memory allocation. Instead, certain operations, by their nature, must perform it. When we discuss LISP in module 5, we will explain what causes implicit allocation of memory from the heap.

B. Type Checking

In module 1, we introduced static semantics and the rules that define whether a program is meaningful. We noted that it is the job of the semantic-analyzer phase of the compiler to check whether a program adheres to these rules. Many of the rules involve type checking. Checking done at compile time is called *static type checking*. When done at run time, it is called *dynamic type checking*. Compiled languages favor the former.

Types must be checked in a variety of different places in the syntax of most languages. Languages that check types in every possible context are said to be *strongly typed languages*.

Type Checking Expressions

The first place that type checking must occur is inside expressions. It must be verified that the types of the operands are valid for that particular operator. In most languages, the remainder operator is defined only for integral types, so attempting to use it in such languages with floating-point operands would be an error.

The other check that must be made is whether, for binary operators, the types of the two operands are compatible. If they are compatible but not identical, type coercion must be performed.

We will elaborate shortly on the way in which type compatibility is determined for user-defined types. For now, let's consider the typical rules for primitive types. In most languages, all numeric types are compatible, and when not identical, the "smaller" type is coerced to the "larger" one.

What varies among languages is whether character types are considered compatible with numeric types. In the C-family of languages, they are considered compatible, but in Pascal and Ada, they are not. A similar variability exists with whether Boolean and enumeration types are considered compatible with numeric types.

When coercion promotes a numeric expression to a "larger" type, usually no unexpected results occur, but some kinds of coercion do produce unexpected results. In C and C++, unsigned types, which we encountered in module 1 as an illustration of orthogonality, are such an example. The rule in those languages is that when unsigned and signed types are mixed, the signed type is converted to an unsigned type, which can lead to unexpected results. Consider the following C++ code segment:

```
signed int s = -1;
unsigned int u = 0;
cout << s + u << endl;
```

The output will be the largest possible unsigned integer. The problem with coercion between signed and unsigned types is that the set of values of neither one is a subset of the other, so no matter which direction coercion is performed, unexpected results can occur.

Type Checking Assignments

Type checking must also be performed on assignment statements. The variable on the left side of the assignment must be checked against the type of the expression on the right. In some languages, the type compatibility rules on assignments are the same as for expressions, which is the case in both C and C++. As a result of this design decision, unexpected results can occur on assignments apart from those associated with unsigned types. Consider the following C++ code segment:

```
int i = 50000;
short s = i;
cout << s << endl;
```

Assuming that the type short is two bytes, the output will be −15536, which is clearly an unexpected result. The problem in this case is that the variable on the left side of an assignment cannot be coerced to another type, and the space reserved for the variable s is not large enough to store 50000. Consequently, the high-order bits are lost, which produces this unexpected result.

When we discuss assignment statements in module 3, we will describe how the rules for type checking on assignments differ in Java.

Type Checking Subprogram Calls

Many languages also perform type checking on subprogram calls. Two checks can be made. The type of the actual parameters can be checked against their corresponding formal parameters. The calls to subprograms that return values are expressions. Consequently, these calls can be placed wherever an expression is allowed, so the return type of the expression can be checked to ensure it is placed in a compatible context.

Failing to perform such type checking of actual against formal parameters can result in logic errors, or worse, may even compromise the compiler's run-time stack at execution time. This lack of type checking was a serious problem in the early versions of C. We will explore how separate compilation is related to this problem and the way in which C++ solves it, when we discuss subprogram parameters in module 3.

Type Checking Control Statements

Type checking may also be appropriate on some control statements. In many languages, such as Ada and Java, the type of the expression embedded in an if or while statement must be Boolean. The switch statement in the C family of languages also must be type checked. The expression in that statement must be an integral type. Languages with case statements often have similar rules.

C. Type Compatibility among User-Defined Types

Until now, in our discussion of type checking, we have discussed only type compatibility among primitive types. Most new languages allow user-defined types, and some allow a greater freedom in defining types than others. Ada allows perhaps the greatest variety of user-defined types. In Ada, the user can define new integer types, real types, enumeration types, array types, record types, access types, and others.

In addition to the wide range of user-defined types, Ada also allows explicit derived types and subtypes. To illustrate this range of possibilities, let's consider three variations of the way in which one might define an integer type for the days in a month. The first approach employs a user-defined integer type. That definition is shown below:

```
type DaysInMonth is range 1..31;
```

The type DaysInMonth is not type compatible with the predefined type Integer or any other user-defined integer type. However, it is compatible with all integer literals within its range.

The second possibility is to define the type explicitly as a derived type, as follows:

```
type DaysInMonth is new Integer range 1..31;
```

The reserved word new indicates an explicit derived type. Like the previous type, it is incompatible with the predefined type Integer or any other user-defined integer type.

The third possibility is to use subtypes as follows:

```
subtype DaysInMonth is Integer range 1..31;
```

When a subtype is used, the subtype is compatible with the original type. Pascal has a similar concept referred to as *subranges*.

Clearly, two variables of different kinds of types are never compatible in Ada, so for example, no array variable is ever compatible with a record variable. Therefore, when compatibility exists, it is only between variables of the same kind. Ada uses

name type compatibility. Because Ada requires almost all types to be named, it is easy to implement name type compatibility. There are, however, some cases in which anonymous types are permitted. Arrays, tasks, and protected objects can be anonymous.
When name compatibility is used and anonymous types are permitted, some additional rules must be defined to accommodate these kinds of types. Consider the following example of arrays with anonymous types:

```
Array1, Array2: array(1..5) of Float;
```

Even in the above situation, Ada considers Array1 and Array2 to be of different types.

In the C family of languages, user-defined types are more limited. In C, structures, unions, and enumeration types can be given names. Arrays and pointer types are not named. Although a typedef statement is included, it does not really create a new type in the sense that Ada type declarations do. C++ added classes to the previous list, although it is not necessarily true that every class name is actually a type name. The names of classes with exclusively static members are not really types. Java dropped structures and unions but added interfaces.

Because many types in C are anonymous, it uses structure type compatibility for the unnamed types, and name type compatibility for the named ones.

One final issue that affects type compatibility is inheritance. In most languages that support inheritance, types in an inheritance hierarchy are compatible with any ancestor type but not any descendant types. The former conversions are often referred to as *upcasts*, and the latter are referred to as *downcasts*.

D. Scope and Lifetime

Although scope and lifetime are related concepts, they are not the same. The scope of a declaration is some portion of the program in which it is declared. *Lifetime* refers to a time period during the execution of a program when the declared variable, constant, and so forth, exist.

Scope Rules

The scope rules of most modern programming languages have their roots in Algol, which was the first block-structured language. Algol and its descendants, Pascal and Ada, allow nested subprograms. None of the languages in the C family do, but all of them allow nested blocks.

In languages that do not permit nested subprograms, such as C, the scope rule for function names is simple. The scope of all function names includes the whole program, so any function in a C program can call any other function in that program. When nested subprograms are permitted, a subprogram can only call other subprograms whose names it can see. Consider the following Ada program:

```
procedure Outer is
    procedure Inner1 is
    begin
        null;
    end Inner1:
    procedure Inner2 is
    begin
        null;
    end Inner2;
begin
    null;
end Outer;
```

In the above program, Outer can call itself and both Inner1 and Inner2. Inner1 can call itself and Outer, but not Inner2. Inner2, like Outer, can call all three.

Regardless of whether the languages support nested subprograms, the scope rules for local names in most languages are the same. The scope of a declaration is from the declaration to the end of the block in which it is declared. Generally, a name is visible through its scope.

Most languages allow a name to be declared in a block and again declared in a block nested inside it. When a name is redeclared in such a fashion, it will not be visible throughout its scope. Consider the following program segment in C or C++:

The Scope of the Outer x (highlighted)	Where the Outer x Is Visible (highlighted)

`{`
`  int x;`
`  {`
`  int x;`
 	
`  x = 1;`
`  }`
`  x = 2;`
`}`
	`{`
	`  int x;`
	`  {`
	`  int x;`
	
	`  x = 1;`
	`  }`
	`  x = 2;`
	`}`

There are other cases where the scope and visibility of identifiers differ, for example, when a class is declared inside a method in a language like Java.

When names are redeclared in this fashion, some languages allow them to be accessed if they are qualified. In Ada, by prepending a name with the block or subprogram name in which it is defined, it is possible to access a name that would otherwise be hidden. In C++, when a global name is hidden by a local of the same name, prepending the name with the scope resolution operator :: makes the global name accessible.

Java is one of the few languages that disallows duplicate local declarations with overlapping scopes as illustrated by the above example. It does allow duplicate local names within the same method if the scopes are disjoint. It also allows instance variables and local variables with the same name inside a class. In that case, the instance variable is hidden by the local variable. Java uses the term *shadowing* to describe such situations.

In Ada, package specifications are separate from package bodies. In C++, function bodies, unless they are inline, are separated from the class definition. In Java, however, the method bodies are embedded in the class definition. Consequently,

the scope rules must be stated in a slightly different way. In a Java class, from the perspective of scope, it is as though all method bodies follow all data declarations and method signatures.

The scope rules that we have described until now are called *static scope rules*. The adjective *static*, as usual, suggests the fact that with such scope rules, scope can be determined at compile time. Although early versions of LISP used dynamic scope rules, few languages use them now. With dynamic scope rules, the local variables of all activation records on the compiler's stack are potentially accessible. Furthermore, depending upon the sequence of subprogram calls, a nonlocal name may reference different activation records if different execution paths are taken.

Lifetime

As we mentioned earlier, the lifetime of a variable is the time period during the execution of the program when it exists. C and C++ are good languages to examine to understand the difference between scope and lifetime. The lifetime of a local variable, which is stack-dynamic by default, is the lifetime of the function in which it is declared. Its life begins when the function is called and ends when the function completes. Consequently, the local variables of a function that is called multiple times have multiple lifetimes. By contrast, global variables, which are allocated as static variables, have lifetimes that are the lifetime of the program.

The connection between scope and lifetime is that local variables have function-wide scope and function-long lifetimes, whereas global variables have program-wide scope and program-long lifetimes. Because of this connection, the distinction between them is sometimes lost. What highlights the difference is the fact that C and C++ allow static local variables, which are a hybrid. They have function-wide scope but program-long lifetimes. Consider the following example in C++.

Move your mouse over the highlighted parts of the method for an explanation of those parts.

```
#include <iostream>
using namespace std;

int global = 0;

void increment()
{
    auto int local_1 = 0;
    static int local_2 = 0;

    cout << global++ << " " << local_1++ << " " << local_2++ <<
}

int main()
{
    for (int i = 1; i <= 4; i++)
        increment();
    return 0;
}
```

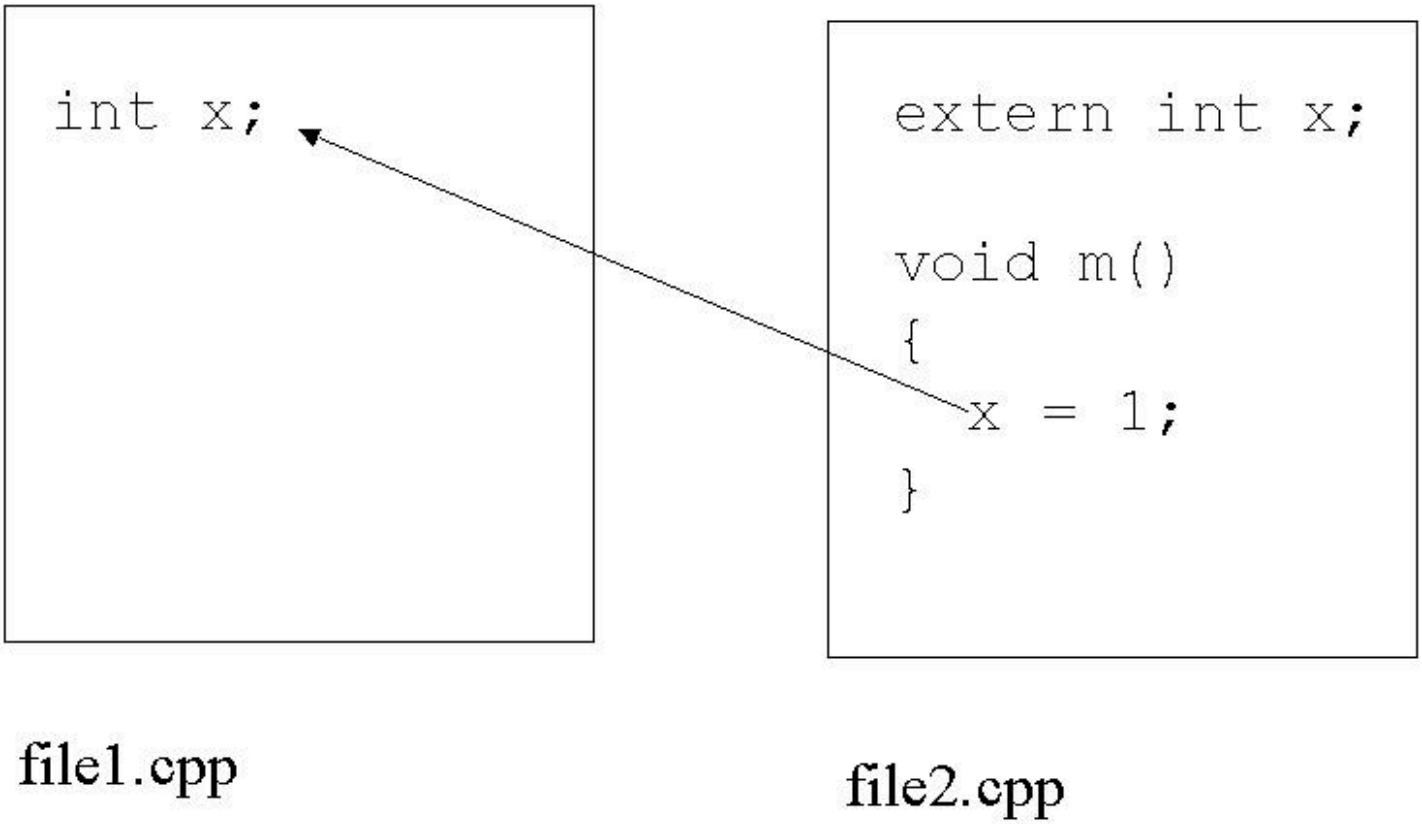
Once you understand the lifetimes associated with each of these three kinds of variables, you should be able to predict the output of this program, which is shown below.

0 0 0
1 0 1
2 0 2
3 0 3

Because global variables and static local variables both have program-long lifetimes, they behave in the same way in this program.

Another aspect of the storage class specifier static in C and C++ warrants comment. Because global variables are allocated by default as static variables, labeling them static would be redundant, much as labeling local variables auto is redundant. Consequently, the designers of C elected to have the storage class specifier static have a different meaning when applied to global variables. To understand its meaning, it is necessary to elaborate on the scope rules for global variables. We had indicated earlier that global variables have program-wide scope. That is true, but global variables can be referenced outside their source file only if they are imported using a declaration containing the storage class specifier extern. Figure 2.1 illustrates how such cross-file references are made.

Figure 2.1
Referencing a Global Variable of a Different Source File



If static is applied to the declaration of x in file1.cpp in the above example, the global variable x will no longer be accessible from file2.cpp. In that case, the linker will generate an error indicating that there is an external reference that cannot be resolved. So, static has the effect of limiting the scope of the global variable.

In C and C++, static applied to local variables extends their lifetime, but when applied to global variables, it limits their scope. We have gone through the effort of explaining this complicated situation primarily because it exemplifies a very poor language-design decision. Ascribing different meanings to the same reserved word in different contexts is never a good language-design choice. The fact that the meaning of static is so difficult to explain underscores this fact. What adds to the confusion is that scope and lifetime are themselves often confused, so having the same reserved words pertain to scope in one context and lifetime in another context compounds the confusion.

If static having two meanings in C were not bad enough, C++ added a third meaning in a third context. When applied to member variables of a class, it specifies that there is only one copy of the variable per class rather than one copy per object, effectively altering its lifetime. Member variables labeled static have a class-long lifetime rather than an object-long lifetime. To its credit, Java adopted this third meaning but dropped the original two.

We conclude with one final example in C that illustrates how important it is to understand variable lifetimes. This function accepts a C string as a parameter and copies it to a local string variable, changing each character to uppercase. It then returns a pointer to that converted string.

```
#include "ctype.h"

#define LENGTH 10

char *to_upper_case(char string[LENGTH])
{
    int index;
    char result[LENGTH];

    for (index = 0; index < LENGTH; index++)
        result[index] = toupper(string[index]);
    return (result);
}
```

No C compiler will produce any compilation errors for the above program. If your compiler is helpful, it may generate a warning, however, indicating that you are returning the address of a local variable. Anyone who would write such a program clearly does not understand the lifetime of stack-dynamic variables. The source of the problem is that this function is returning a pointer to a local string whose lifetime will end as soon as the function completes.

The behavior of such functions would confuse anyone who does not understand the way stack-dynamic variables are allocated. If the calling function immediately displayed what was returned, the function would appear to work because stack-dynamic variables are not erased when their lifetimes end. But if another function were called and afterward the converted string was displayed, garbage would most likely be output. In that case, the local string would have been overwritten by the local variables of the function last called.

II. Scalar Data Types

Now that we have completed our discussion of the issues related to data types, which we called *data control*, we turn our attention to the specific data types that are included in most programming languages. We begin with scalar data types,

which are typically predefined.
As is the case with so many of the comparisons we make with imperative languages, the approach taken by the C family of languages differs with regard to scalar data types from that taken by Ada and its ancestors. So, in the discussion that follows, we will explain both approaches.

A. Integer Types

In the C family of languages, there is a collection of predefined integer types of different sizes. As we mentioned earlier, in C and C++, these sizes are machine independent, but in Java, the language defines them. In C and C++, the integer types include short, int, and long. Java adds byte to that list. In all three languages, char can be used for integer data.

Although Ada also has predefined integer types of different sizes, as we saw earlier, it is possible to define new derived and subtypes for integer data. This ability allows the programmer to distinguish between integers used for a specific purpose and integers in general. In our earlier example, making DaysInMonth a derived type—either implicitly or explicitly—makes it type incompatible with the predefined type Integer and any other integer type. Both derived types and subtypes make run-time range checking possible, which improves the reliability of programs. Ada does not allow the type Character to be used for numeric data.

B. Real Types

The approach taken for real-valued numeric data in the C family is much the same as the approach taken for integer data. It provides several predefined sizes—all stored in floating-point format. C and C++ have three sizes: float, double, and long double. Java includes only the first two.

Ada's approach is quite different. First, Ada allows both floating-point and fixed-point representations. Although there are predefined floating-point types, the programmer is allowed to define new floating-point types unique to a particular application. An example of such a user-defined floating-point type definition is shown below:

```
type MyFloatType is digits 10 range 0.0..100000.0;
```

Floating-point representations allocate some number of bits for the mantissa and some for the exponent. The digits qualifier specifies the amount of required precision. By multiplying the request digits by the number of bits needed per digit, which is between 3 and 4, the number of bits needed for the mantissa can be computed.

In a similar fashion, the range qualifier could be used to compute the number of bits required for the exponent. Although this is an interesting language feature, user-defined floating-point types have limited practical value because, to be efficient, the floating-point type must be implemented in the hardware. The best the compiler can do with such information is to select the smallest hardware-defined floating-point type that is large enough to accommodate this definition. Having the compiler make the selection, however, does avoid problems with the portability of programs between machines with different floating-point hardware implementations.

Ada also allows fixed-point types to be defined. C, C++, and Java have no comparable types, but C# does have a type called decimal that we will discuss shortly. Let's consider an example of a fixed-point type in Ada:

```
type Dollars is delta 0.01 range 0..100_000;
```

The delta qualifier specifies the required amount of accuracy. One can determine how many bits are needed to the right of the decimal to provide this level of accuracy.

A key difference between fixed- and floating-point types is how the model numbers for such type definitions are distributed along a number line. With fixed-point types, they are evenly distributed. Suppose, for example, the fixed point were placed with four bits to the right. A model number would appear at every $1/16^{th}$ mark along the number line. With floating-point types, the model numbers are closely clustered near zero and more widely separated the farther from zero they are.

An inherent problem with any binary representation of real numbers, whether fixed or floating point, is the possibility of representation errors. This problem is particularly troublesome in business applications. One way to avoid such representation errors is to use binary-coded decimal.

Not surprisingly, COBOL has a decimal type, which uses BCD, as does C#.

C. Nonnumeric Types

Most languages provide some predefined scalar types for nonnumeric data such as character data. We have already observed how some languages allow character data to be used for integer data, but other languages do not.

Boolean Data

The other predefined scalar data type is Boolean. Again, some languages differentiate between integer data and Boolean data; others do not. In C and C++, the type bool is just a renaming of int. An integer value can be used as Boolean data, with 0 being treated as false and everything else being treated as true. Java, Ada, and its ancestor languages, by contrast, have true Boolean types.

Not distinguishing between Boolean and integer data can lead to some unfortunate misinterpretations of C and C++ programs. The following if statement in C++ illustrates this problem:

```
if (1 <= x <= 10)
    cout << "Between 1 and 10";
```

In this case, the attempt is to determine whether x is between 1 and 10, but the expression in the if condition will always evaluate to true because the first subexpression:

```
1 <= x
```

will evaluate to 0 if it is false and evaluate to 1—the nominal representation for true—otherwise. In either case, that result is less than or equal to 10, so the expression as a whole is true.

Experienced programmers know the proper way to write the intended expression is:

```
1 <= x && x <= 10
```

What makes this misinterpretation so troubling is the fact that writing 1 ≤ x ≤ 10 is perfectly good mathematical notation for expressing the desired condition. So, we have a situation that we previously described as one of the worst results in language design—the ability to write something that would be reasonably expected to mean one thing and have it be interpreted to mean something else.

In languages that have a true Boolean type, such expressions produce semantic errors—specifically, a type incompatibility between a Boolean and an integer.

Enumeration Types

In addition to providing these two predefined nonnumeric types, many languages allow the user to define nonnumeric types called *enumeration types*. In Ada, the boolean and character types are consider predefined enumeration types.

Pascal was among the first languages to allow enumeration types, so it is not surprising to find them in Ada. Although they were not a part of the earliest versions of C, they were subsequently added and, therefore, are also in C++. They were dropped originally in Java and only recently added in version 5.0.

Allowing the user to define an enumeration type has several advantages. It allows names to be used for the literals in place of numbers. One could argue that the same advantage could be achieved using named constants, which is true. However, a second advantage that simply naming constants does not achieve, is conveying that some set of constants constitutes a type. The final advantage is that checking can be done to ensure that variables of such types are never assigned improper values. Not all languages that include enumeration types provide the last advantage. C and C++ do not. The following program segment illustrates this fact:

```
enum PrimaryColors {RED, YELLOW, BLUE};
PrimaryColors color = (PrimaryColors) 10;
```

Although enumeration types are not implicitly compatible with integers, an integer can be type cast to any enumeration type, which is what is being done in the above code. Neither a compilation nor run-time error results, yet the variable color will contain an improper value.

In Ada, such explicit conversion of integers to enumeration types is prohibited. This lack of type-safety is likely one reason that Java initially chose to omit enumeration types. When Java incorporated enumeration types into the language in version 5.0, it was done in a way that is type-safe. In Java, enumeration literals are objects, not integers. Every enumeration type is implicitly derived from the predefined class Enum. Because enumeration literals are objects, it is possible to include user-defined data and methods. A Java enumeration type definition for primary colors, which illustrates these capabilities, is shown below.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
import java.awt.*;
enum PrimaryColors
{
    RED(Color.RED),
    YELLOW(Color.YELLOW),
    BLUE(Color.BLUE);

    private Color color;

    private PrimaryColors(Color color)
    {
        this.color = color;
    }
    public Color getColor()
    {
        return color;
    }
}
```

All enumeration types in Java inherit several methods from Enum. The first is a method name, which returns the enumeration literals as a string, so RED.name() returns "RED". The second is ordinal, which returns an integer corresponding to the position of the literal in the type definition—RED.ordinal() being 0. Enum implements the Comparable interface so a compareTo method is defined that compares literals based on their ordinal values. Finally, a method values returns an array of all literals, which is useful in the *for-each* style for loop that was added in version 5.0 and that we will discuss in module 3.

Although Ada does not allow the user to associate new data or subprograms with enumeration types, there is a technique for accomplishing what most of the aforementioned methods inherited from Enum provide. We shall now discuss this technique in the broader context of discrete types.

D. Type Attributes

Associated with Ada's categorization of data types is the syntax of type attributes. Each category of types has an associated set of attributes. One such category is discrete types. Other languages sometimes refer to such types as *ordinal* or *integral* types. Included in the discrete-type category are integer types and enumerated types. Attributes can be either constants or subprograms. Let's consider the Ada implementation of our previous enumeration type of primary colors as an example:

```
type PrimaryColors is (Red, Yellow, Blue);
```

For this type, there is an attribute constant PrimaryColors'First whose value is Red. For every discrete type, the attribute 'Image is a function that returns the string representation of the enumeration literal, so for example, PrimaryColors'Image(Red) returns "Red". Similarly, there is an attribute function 'Pos that returns the position or ordinal value—PrimaryColors'Pos(Red) returns 0.

In Ada, each type category has an associated set of attributes for all types belonging to that category. These attributes are provided by the compiler and do not have to be defined by the programmer. In our upcoming discussion of arrays, we will see some of the attributes defined for array types.

III. Structured Data Types

Next, we consider structured data types, which are data types whose variables contain a collection of values. Arrays and records are the two fundamental kinds of structured data types. Both appeared early in the history of programming languages—Fortran being the first major high-level language to support arrays and COBOL being the first to support records.

A. One-Dimensional Arrays

Arrays are collections of elements of the same type. Although they may seem more restrictive than records that allow collections of different types, their generality lies in their selection mechanism. Any collection must have some mechanism for selecting individual elements within that collection. Subscripting or indexing is the technique used by arrays. What makes this selection mechanism so powerful is that array subscripts can be expressions that contain variables.

This flexibility does have an unfortunate consequence, though: the subscripts can be outside the defined bounds of the array. One important design consideration of any language that supports arrays is whether bounds checking on subscripts will be performed. Because subscripts can be expressions that contain variables, if this checking is done, it must be done at run time. Any such run-time checking adversely affects the execution speed of a program. Failing to make such checks, however, can lead to highly unreliable programs. C and C++, which emphasize speed, do not perform such checking. Ada and Java, which value reliability above all, do perform subscript-bounds checking.

Array Allocation

Like any variables, array variables can be allocated in static memory, on the compiler's run-time stack, or on the heap. With array variables, another consideration is whether the arrays' bounds are values that can be determined at compile time. With statically allocated arrays, the bounds must be determinable at compile time, and with arrays allocated on the heap, it makes little difference whether or not they are determinable at compile time.

It is with arrays allocated on the stack that languages differ. In C and C++, the size of stack-dynamic arrays must be able to be determined at compile time. Consequently, the following attempt to write a function in C++ that reverses an array by using a local copy fails to compile.

```
void copy(int array[], int length)
{
    int copy[length];

    for (int i = 0; i < length; i++)
        copy[i] = array[i];
    for (int i = 0; i < length; i++)
        array[i] = copy[length - 1 - i];
}
```

The error produced by many compilers is a message that indicates that the size of an array must be a constant, which leads many novices to modify the function heading as follows:

```
void copy(int array[], const int length)
```

That modification does not fix the error because by *constant*, the error means the size of an array must be able to be determined at compile time—not that it cannot have its value changed. Allowing such arrays is undesirable because it is difficult for a compiler to calculate the address of local variables relative to the beginning of the activation record—a process we described earlier in this module when we introduced stack-dynamic allocation. Such calculations can be made only if the size of the variables is known.

Ada does allow stack-dynamic arrays to contain variables. Our above failed attempt to write such a function in C++ can be accomplished in Ada.

Move your mouse over the highlighted parts of the procedure to see the explanations of those parts.

```
procedure Main is
    type IntegerArray is array (Integer range <>) of Integer;

    procedure Copy (Original: in out IntegerArray) is
        Copy: IntegerArray(Original'Range);
    begin
        for I in Original'Range loop
            Copy(I) := Original(I);
        end loop;
        for I in Original'Range loop
            Original(I) := Copy(Original'Length - 1 - I);
        end loop;
    end Copy;

    Original: IntegerArray(1..5);
begin
    Original := (1, 2, 3, 4, 5);
    Copy(Original);
end Main;
```

Array Aggregates

Next, we consider the extent to which various languages support array constants or array aggregates. In the C family of languages, array aggregates can be used for initialization only when an array is declared. In C or C++, we can write:

```
int array[] = {1, 2, 3, 4, 5};
```

But if we attempt to use the aggregate in an assignment, a compilation error will result, as seen below:

```
int array[5];
array = {1, 2, 3, 4, 5}; // does not compile
```

With strings that are character arrays, such assignments are possible if the array aggregates are written as string literals.

In our previous Ada example, we saw that Ada was more flexible. It allows array aggregates to be used in assignments. It provides another variation on the array aggregate syntax, called *positional association*. The array aggregate assignment used in the previous program could be written using positional association as follows:

```
Original := (5 => 5, 1 => 1, 4 => 4, 3 => 3, 2 => 2);
```

When positional association is used, order is no longer important. Ranges, choices, and the reserved word others adds to this flexibility, as illustrated in the following example:

```
Mixture: IntegerArray(1..50) := (1..10 => 1, 20 | 30 | 40 => 2,
    others => 0);
```

In the example above, the values in subscripts 1 through 10 are initialized to 1, the values in subscripts 20, 30, and 40 are set to 2, and all the other subscripts are initialized to 0.

Array Assignment and Operations

Another area in which there is considerable variation among languages is on the number of operations that are predefined on whole arrays.

Let's first examine assignment. In C and C++, attempts to assign one whole array to another result in a compilation error as the following code illustrates:

```
int array1[] = {1, 2}, array2[] = {3, 4};
array1 = array2;
```

The assignment is not permitted because array names are treated as constant pointers, which cannot be modified. The situation in Java is different, however. Array assignments are permitted, but like any object assignment, they are shallow copies.

Whole array assignments are permitted in Ada. The following lines illustrate such an assignment.

```
Array1: IntegerArray(1..2) := (1, 2);
Array2: IntegerArray(2..3) := (3, 4);
Array1 := Array2;
```

The rule on assignment requires that the types and lengths match, as is true in the above example. Unlike Java, the copy is deep—the actual values are copied.

None of the languages in the C family have any other predefined operations on whole arrays. Ada does, however. It permits concatenation, slicing and comparison for equality, and inequality for one-dimensional arrays.

B. Multidimensional Arrays

One fundamental language design issue regarding multidimensional arrays that language designers must consider is whether to implement multidimensional arrays as rectangular arrays or arrays of arrays or both.

Rectangular Arrays

When a language implements multidimensional arrays as rectangular arrays, a block of memory is allocated whose size is the product of the length of each dimension multiplied by the size of each array element.

When array references are made, it is necessary for the compiler to generate code that will compute the position of each array element. That code must compute the location of the array element using a *storage allocation function* that maps two dimensions in one. Two kinds of mappings—*row-major* and *column-major*—are typically used. Figure 2.2 illustrates how the elements of a two-dimensional array with two rows and three columns would be enumerated according to both orders.

Figure 2.2
Example of a 2 × 3 Array
Enumerated in Row-Major and Column-Major Orders

Example Of A 2 × 3 Array Enumerated in Row and Column Major

Row Major Order

0	1	2
3	4	5

Column Major Order

0	2	4
1	3	5

The storage allocation function for row-major order is shown below, assuming array subscripts always begin at 0, as they do in the C family of languages:

$\Lambda(i, j) = (i \times \#\text{columns} + j) \ \text{size}$

Making the lower bound 0 avoids having to subtract it from the corresponding subscript. Notice that the number of columns is involved in the calculation. When we discuss array parameters in module 3, we will explain how that affects multidimensional array parameters in C and C++.

Most languages that implement rectangular arrays use row-major order. Fortran is the one exception. The formula for column major, shown below, is similar to that for row major, but instead involves the number of rows.

$\Lambda(i, j) = (j \times \#\text{rows} + i) \times \text{size}$

Arrays of Arrays

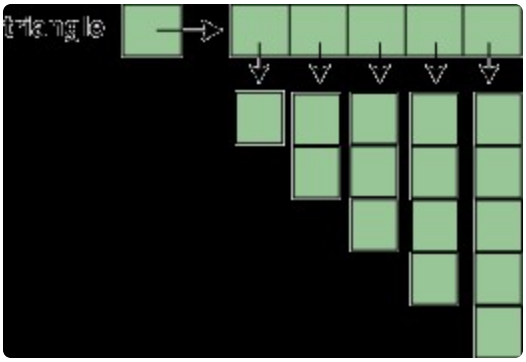
In some languages, such as Java, multidimensional arrays are exclusively arrays of arrays. In most languages that support arrays of arrays, those arrays can be ragged. In languages that do not explicitly support ragged arrays but do contain pointers, ragged arrays can be created using an array of pointers.

To illustrate this idea, consider the following two-dimensional Java array that creates a triangular array. The first row has one column, the second has two, and so on.

```
int [][] triangle = new int[5][];
for (int row = 0; row < triangle.length; row++)
    triangle[row] = new int[row + 1];
```

Figure 2.3 illustrates the ragged array that is created by the above code.

Figure 2.3
A Ragged Array



In the previous example, as with any two-dimensional array, triangle.length refers to the number of rows. triangle[0].length refers to the number of columns in the first row.

Some languages support both rectangular arrays and arrays of arrays. Ada and C# are such languages. In Ada, the arrays of arrays cannot be ragged.

C. Strings

We have elected to treat strings separately from arrays because in some languages, strings are actually arrays, and in other languages, they are objects. In C, strings are arrays of characters with the special provision that they are terminated with the null character. String operations in C are accomplished with calls to predefined functions. Many of these operations, such as strcpy, which is used to deep copy strings, are prone to cause memory to be overwritten. Such overwriting occurs when the destination string is smaller than the source string. Because no size information is kept at run time for any C arrays—including strings—it is impossible for such functions to check whether sufficient space has been allocated.

C++, being a superset of C, includes the C-style strings but also has a predefined class named string, which is far safer to use. Because C++ allows operator overloading, the assignment operator, all relational operators, and the + operator are all overloaded in the string class. The + operator is used to define concatenation.

Furthermore, C++ treats [] as an operator, and the string class overloads that operator, so that strings can be subscripted like arrays. It is even possible to put a subscripted string variable on the left-hand side of an assignment. In module 3, we will explain what feature of C++ enables this unusual ability, in effect, to place a function call on the left side of an assignment.

As elegant as the C++ implementation of the string class is, there is one shortcoming—C++ must also support C strings. Consequently, the type of string literals in C++ is char*, not string. This fact leads to some unfortunate misinterpretations. If two string literals are compared with any of the relational operators, this comparison is treated as pointer comparison, not a call to the overloaded relational operator defined in the string class. Shown below is an even more obscure example of the kind of misinterpretation that can result:

```
string s = "a" + 'b';
```

This is another example of the worst possible interaction of various language features that produce a program that appears to do something reasonable but is misinterpreted in a highly unreasonable way. Most programmers would expect s to contain the string "ab"—the result of having concatenated the string literal "a" with the character literal 'b'. Instead, what results is garbage.

One must understand a number of features of C++ to explain how this statement is interpreted. First, as we mentioned, string literals in C++ have type char*—not string. Next, characters are treated as integers, so the character literal is interpreted as the integer corresponding to its ASCII value. Next, the + operator is interpreted as pointer arithmetic, not concatenation—a topic we will discuss shortly. That pointer arithmetic produces an address viewed as type char*. To understand why the types of opposite sides of the assignment are not incompatible, one must be aware of yet another feature of C++—the fact that certain constructors may be implicitly called to perform type coercion. The fact that the string class has a constructor with the following prototype:

```
string(char *);
```

explains why this assignment is not rejected as a static semantic error.

Although all the languages in the C family often implement features in the same way, with strings, that is not the case, so Java's approach differs from that of both C and C++. First, in Java, the type of string literals is String. Consequently, the Java code equivalent to our earlier C+ code shown below behaves as expected.

```
String s = "a" + 'b';
```

Unfortunately, Java does not support operator overloading, so assigning Java strings results in shallow copies. Fortunately, Java strings are immutable, so shallow copies do not cause problems. Problems do arise with the inability to overload the relational operators. The compareTo method must instead be used in much the same way strcmp is used with C strings. Furthermore, when comparing strings, if the == operator is mistakenly used, a shallow comparison results, which is seldom what is desired.

In Ada, the predefined type String is an unconstrained array type of characters. Because strings are arrays, all array operations are defined for strings, which include subscripting, slicing and comparing for equality, and inequality. Because Ada permits operator overloading, the remaining four relational operators can be overloaded for strings.

D. Records

Records are the other fundamental structured data type. As we mentioned earlier, they are more general than arrays in the sense that they can collect data elements of different types. But because the components are named, not numbered, the selection mechanism is less powerful. Most languages use the dot notation for selecting a record component. Because names are essentially constants, every record component reference can be resolved at compile time, unlike subscripted arrays.

As we noted in our introduction to structured data types, COBOL was the first language to include records. Except for early versions of Fortran and more recently Java, most other languages have them. In both Pascal and Ada, these structured data types are called *records*, but in C, C++, and C#, they are referred to as *structures*, using the reserved word struct to define them.

It is worthwhile to explore the evolution of structures in the C family. In C, which is not object-oriented, structures are limited to data, making them true records. In C++, which is object-oriented, structures and classes are nearly identical—both allowing data members and member functions. The only difference between structures and classes is whether the members are public or private by default.

We have been emphasizing how important the use of reserved words is in highlighting important language features. C++'s use of two different reserved words—struct and class, is a poor design because the meanings of these two reserved words are so similar. Leaving structs as they were in C would have been a better language-design choice. In C#, by contrast, structures and classes are fundamentally different.

The designers of Java made the decision to drop structures altogether. Because a class can have only data and no methods, it can be reduced to a simple record structure. Of course, in well-designed object-oriented programs, having a class with only data but no methods is seldom a good design unless, perhaps, the class is nested inside another class.

E. Unions and Variant Records

C and C++ have unions, which syntactically look very similar to structures. What they represent is different, however. A whole structure variable contains *all* the components, whereas a whole union variable contains *only one* of the components. The union implementation in these languages is referred to as a *free union*. It is one area where no type checking is performed and the type system can be compromised. The following code segment illustrates how the types can be compromised.

```
union
   {
       int i;
       double d;
   } u;

   u.d = 1.0;
   cout << u.i;
```

The code above outputs garbage, yet no error is produced.

Another kind of union is a *variant record*, also called a *discriminated union*. In a variant record, a tag field or discriminant contains a value indicating which variant is applicable. Pascal has variant records, but types can still be compromised because the tag field, as it is called in Pascal, can be changed without changing the remaining fields, producing a problem similar to the one above in C++.

Ada solves this problem. Consider the following variant-record definition in Ada. Move your mouse over the highlighted parts of the code segment to see the explanations of those parts.

```
procedure Main is
    type MaritalStatus is (Single, Married);
    type Person(Status: MaritalStatus := Single) is
    record
        Name: String(1..4);
        Age: Integer range 0..100;
        case Status is
            when Married =>
                Spouse: String(1..4);
            when Single =>
                null;
        end case;
     end record;

    John: Person := (Status => Single, Name => "John", Age => 2
begin
    -- John marries Jane
    John := (Status => Married, Name => "John", Age => 25,
        Spouse => "Jane");
end Main;
```

Ada allows the discriminant to be changed only for unconstrained variant records. Furthermore, when the discriminant is changed, all the record components must be changed as is done in the above example.

Neither Java nor C# has either unions or variant records because inheritance is the approach that would be used in those languages to implement such data relationships. In the above example, SinglePerson and MarriedPerson would be classes derived from a base class Person.

IV. Pointers and References

Any language that has explicit heap-dynamic memory allocation must have either pointers or references—sometimes both.

A. Declarations

We begin our discussion with an illustration of the syntax for declaring pointers and references.

We chose C++ to illustrate declarations because it is unusual among languages in that it has both pointers and references. Both must be explicitly declared as illustrated below.

Move your mouse over the highlighted parts of the code segment to see the explanations of those parts.

```
int x = 5;
int &rx = x;
int *px = &x;
```

C, by contrast, supports only pointers, as do both Pascal and Ada. Ada calls them *access types* and uses the reserved word access in their type declarations. Java has only references, but unlike C++, they are implicit references and apply only to objects.

B. Operations

Next, let's consider the operations that typically accompany pointers. References have no associated operators, regardless of whether they are explicit as in C++ or implicit as in Java.

Dereference and Address Operators

The one operation that every language that has pointers needs is a dereference operator. In Pascal, the symbol ^ is used for this operator. In C and C++, the symbol * is used. Ada uses a more unusual syntax—.all—which can be omitted when

dereferencing pointers to arrays or records.
Less common is the inverse of the dereference operator—the address operator—that creates a pointer from an ordinary variable by providing its address. Such operators are present only in languages such as C or C++ that allow pointers to point to stack-dynamic variables. In those languages, the symbol for that operator is &. We saw that operator used in the pointer initialization in our previous example.

We have commented on the undesirable practice of using the same reserved word to mean different things in different contexts. In C++, the symbols * and & are used in a similar fashion. They are overloaded in a way that their meaning as unary operators is unrelated to their meaning as binary operators. As unary operators * and & mean *dereference* and *determine address*, respectively. As binary operators, they represent *multiplication* and *bitwise and*, respectively.

Compounding the overuse is that these symbols are also used in type specifiers to specify *pointer* and *reference* types, respectively. In that latter context, they are not ordinary operators, but type operators. One might argue that their uses as type modifiers and unary operators are related, but they are certainly not identical. Needless to say, such overuse of symbols can make it challenging to master the syntax of pointers and references in C++.

Pointer Arithmetic

Pointer arithmetic is an unusual feature to find in languages that have pointers. C and C++ are the only popular high-level languages that contain this feature. Pointer arithmetic is most useful with array parameters or dynamically allocated arrays. Because the array and pointer notation is interchangeable in C and C++, one can avoid pointer arithmetic entirely. A common practice in C was to use pointer arithmetic with array parameters to improve the efficiency of a function by eliminating the need for the multiplication that is generated whenever a subscripted array element is accessed.

To understand what multiplication is eliminated, examine the following translation that the compiler must make whenever it encounters a subscripted one-dimensional array element:

`a[i]` is translated by the compiler to `a + i × sizeof`(the array elements)

Consider the following three equivalent implementations of a simple function to sum all the elements of an integer array.

First, consider the version in which the parameter is declared and referenced as an array. Move your mouse over the highlighted part of the function to see an explanation of that part.

```
int sum_array_1(int array[], int length)
{
    int sum = 0;
    for (int i = 0; i < length; i++)
        sum += array[i];
    return sum;
}
```

Next, let's look at the same function rewritten declaring the parameter as a pointer. Move your mouse over the highlighted part of the function to see an explanation of that part.

```
int sum_array_2(int* array, int length)
{
    int sum = 0;
    while (length-- > 0)
        sum += *array++;
    return sum;
}
```

We provide the third version to illustrate subtraction of one pointer from another. Move your mouse over the highlighted part of the function to see an explanation of that part.

```
int sum_array_3(int* array, int length)
{
    int sum = 0;
    int* start = array;
    while (array - start < length)
      sum += *array++;
    return sum;
}
```

It is worth noting that a good optimizing compiler can perform these programmer-encoded optimizations.

The three pointer-arithmetic operations are summarized in table 2.1.

Table 2.1
The Three Kinds of Pointer-Arithmetic Operations

Operation	Computation
pointer + integer	pointer + integer × size
pointer - integer	pointer - integer × size
$pointer_1$ - $pointer_2$	($pointer_1$ - $pointer_2$) / size

Because the behavior of pointer arithmetic depends on the type of the object being pointed to, pointers to different types are not type compatible in C and C++.

The fact that subtraction of pointers is defined but addition is not defined may seem strange at first. There are other instances, however, in which only the three arithmetic operations in the table above are meaningful, for example the arithmetic of time and addresses. The correspondences are illustrated in table 2.2.

Table 2.2
Correspondence of Pointer, Time, and Address Arithmetic

Pointer Arithmetic	Time Arithmetic	Address Arithmetic
pointer	time	relative address
integer	duration	absolute address

In the case of time, subtracting two times produces a duration, not a time. Adding two times is meaningless. If you have studied assembly-language programming, you understand the difference between relative and absolute addresses. The arithmetic of absolute and relative addresses behaves in the same way as pointer arithmetic. Subtracting two relative addresses produces an absolute address, but adding two relative addresses is meaningless.

C. Dangling References and Garbage

Now let's consider some problems that can arise in any language that has pointers. As we noted earlier, pointers in C and C++ can point to variables that are allocated on the stack. This capability can lead to problems like the one we illustrated earlier in our discussion of lifetime. In that case, the programmer returned a pointer to a local variable, which created a dangling reference or dangling pointer.

Dangling pointers can be created in other ways in C and C++ as well. Consider the following code segment:

```
int *x;
*x = 5;
```

Although the above code is seriously in error, no error is generated at compile time. At run time, if an error occurs, it is likely to be generated by the operating system—most likely a memory-protection error—although there is certainly no guarantee that any error will result. The problem in this case is that the pointer has never been initialized, yet it is being dereferenced.

There is yet a third way to create a dangling pointer in C and C++, which is illustrated by the following C++ code segment:

```
int *px = new int, *py = px;
delete px;
```

At this point, the pointer py is left dangling.

A related issue concerning pointers and any memory allocated on the heap is the possibility of creating *garbag*e. Another term that is less often used but is perhaps more descriptive is *dangling object*. Figure 2.4 illustrates how easily garbage can be created.

Figure 2.4
Creating Garbage

0:00 / 0:30

In a language such as C or C++ that does not have automatic garbage collection, it is the responsibility of the programmer to ensure that garbage is not created. To prevent the formation of garbage, explicit deallocation is needed. The C++ code segment in figure 2.4 is shown below. The command to explicitly deallocate the memory is highlighted.

```
int *p = new int;
int *q = new int;
*P = 5;
*q = 6;
delete p;
p = q;
```

In languages that do implement automatic garbage collection, such as Java and C#, the user is freed from this responsibility, and garbage will never accumulate endlessly.

D. Pass by Reference

One benefit of allowing pointers to point to the compiler's run-time stack is that they can be used to pass parameters by reference as illustrated in the following C program.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <stdio.h>

void swap(int *value1, int *value2)
{
    int temp;

    temp = *value1;
    *value1 = *value2;
    *value2 = temp;
}

void main()
{
    int value1 = 10,value2 = 20;

    swap(&value1, &value2);
    printf("value1 = %d, value2 = %d\n", value1, value2);
}
```

Including the syntax of references in C++ does simplify significantly the syntax of pass by reference when references are used in place of pointers. Consider the previous C program, rewritten in C++, using reference parameters in place of pointers.

Move your mouse over the highlighted parts of the program to see the explanations of those parts.

```
#include <iostream>
using namespace std;

void swap(int &value1, int &value2)
{
    int temp;

    temp = value1;
    value1 = value2;
    value2 = temp;
}

int main()
{
    int value1 = 10,value2 = 20;

    swap(value1,value2);
    cout << "value1 = " << value1 << " value2 = " << value2 <<
    return 0;
}
```

Another interesting consequence of including references in the language is the ability to write functions that return references. In module 3, when we discuss subprograms, we will give some examples of functions that return references and discuss how useful this feature can be.